CPSC 457 - Assignment 3

Memory Allocation Reflection

Elyas Hossaini

The most challenging part of the implementation was keeping track of the selected hole and its predecessor while scanning the list. The predecessor at the end of a search may not belong to the chosen hole, so each algorithm saves both pointers when it finds a better candidate. This allows an exact-fit allocation to remove the correct entry, including when that entry is the head of the list. Partial allocations instead advance the selected hole's base and reduce its length, preserving the list's address order. Best-fit chooses the smallest hole large enough for the request. Compared with Minix's default first-fit policy, it minimizes the immediate leftover space in the chosen hole and can preserve larger holes for later requests. However, it usually searches the entire list unless it finds an exact fit, and repeated allocations can leave tiny fragments that are difficult to reuse. First-fit can stop at the first suitable hole, so it generally requires less searching. Worst-fit chooses the largest suitable hole. Its advantage is that the remaining portion may still be large enough to serve another request, while smaller holes remain available. Its disadvantages are that it must inspect every eligible hole and can consume the large contiguous regions needed by future large requests. Neither policy is guaranteed to reduce overall fragmentation compared with first-fit; the outcome depends on allocation sizes, lifetimes, and how freed regions merge. Tests covered partial allocations, exact fits at every list position, equal-size ties, insufficient memory, and repeated allocation/free cycles. Both policies were also compiled and booted in Minix with hole-list diagnostics enabled to check their allocation choices.